

Proyecto Final – Teoría Estadística (XS3310)

Universidad de Costa Rica · Ciclo I 2026

Anderson Martínez Abarca, Sebastián Calvo, Justin Zhu, Fabián Almirall

02 de July de 2026

Contents

1	Parte I. Comparación de intervalos de confianza utilizando diferentes técnicas	1
1.1	Ejercicio 1. Análisis descriptivo de la población	1
1.2	Ejercicio 2. Muestra aleatoria y estimación kernel	2
1.3	Ejercicio 3. IC exacto con varianza conocida ($\sigma^2 = 100$)	4
1.4	Ejercicio 4. IC asintótico con varianza desconocida	4
1.5	Ejercicio 5. IC bootstrap estándar ($B = 1000$, sin librería <code>boot</code>)	4
1.6	Ejercicio 6. Comparación de los tres intervalos	5
2	Parte II. Estudio de simulación	5
2.1	Ejercicio 1. Objetivo del estudio de Monte Carlo	5
2.2	Configuración	6
2.3	Ejercicio 2. Resultados con medidas de variabilidad	7
2.4	Ejercicio 3. Comparación de coberturas y efecto de la semilla	7
2.5	Ejercicio 4. Diseño de una simulación más completa	8

1 Parte I. Comparación de intervalos de confianza utilizando diferentes técnicas

El archivo `población.RData` contiene un vector de altura (en centímetros) de una población de 1500 estudiantes universitarios en Costa Rica. El objetivo es extraer una muestra de tamaño $n = 50$ de esta población con reemplazo y realizar inferencia sobre la altura promedio poblacional, denotada por θ . Específicamente, se busca el intervalo de confianza del 99% para θ utilizando tres métodos.

```
# Renombra el archivo para evitar problemas de encoding con la 'o' acentuada,  
# luego carga el vector x con las alturas de los 1500 estudiantes universitarios  
load("población.RData")
```

1.1 Ejercicio 1. Análisis descriptivo de la población

```
# Histograma de densidad (probability = TRUE) para poder superponer la curva normal  
hist(x,  
  probability = TRUE,  
  col = "lightblue",  
  main = "Histograma de Alturas vs. Curva Normal",  
  xlab = "Alturas (cm)",  
  ylab = "Densidad")
```

```

# Secuencia de valores para evaluar la curva normal teórica
Z2 <- seq(min(x), max(x), length = 1500)
# Superpone la curva normal usando la media y desviación estándar poblacionales
lines(Z2, dnorm(Z2, mean = mean(x), sd = sd(x)), col = "red", lwd = 2)
# Línea vertical en la media poblacional
abline(v = mean(x), col = "darkblue", lwd = 2, lty = 2)
legend("topright",
      legend = "Media poblacional",
      col = "darkblue", lty = 2, lwd = 2, bty = "n")

```

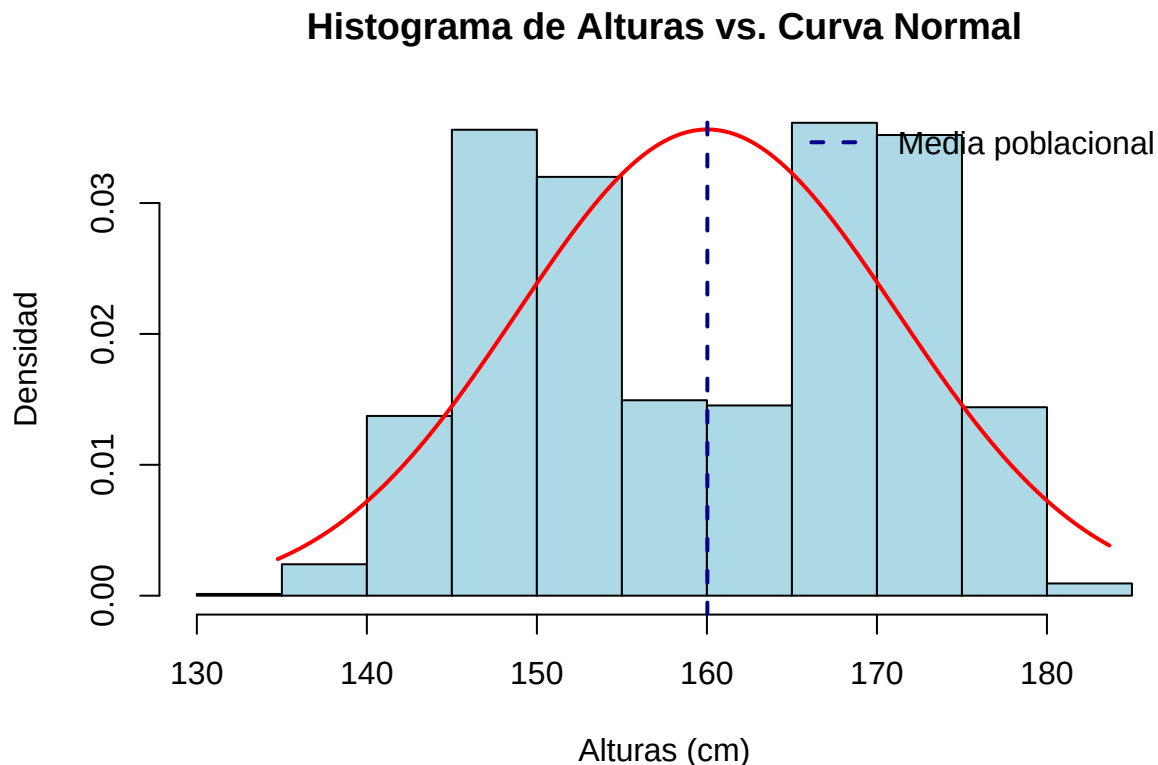


Figure 1: Distribucion de alturas de la poblacion (N = 1500).

El histograma no muestra una forma claramente normal — las barras no forman una campana definida y presentan irregularidades que dificultan concluir normalidad a partir del gráfico. Sin embargo, dado que se trata de alturas de personas y que esta variable típicamente sigue una distribución aproximadamente normal en poblaciones humanas, **es razonable asumir normalidad como aproximación de trabajo** para efectos del análisis.

1.2 Ejercicio 2. Muestra aleatoria y estimación kernel

```

# Fija la semilla para reproducibilidad
set.seed(1117)
# Extrae una muestra aleatoria de n = 50 con reemplazo de la población x
n <- sample(x, 50, replace = TRUE)

```

```

# Estima la densidad kernel gaussiana con ancho de banda h = 1 (más rígido)
densidad_h1 <- density(n, kernel = "gaussian", bw = 1)
# Estima la densidad kernel gaussiana con ancho de banda h = 5 (más suave)
densidad_h5 <- density(n, kernel = "gaussian", bw = 5)

# Histograma de fondo para contextualizar las curvas kernel
hist(n, probability = TRUE, col = "gray90", border = "white",
     ylim = c(0, max(densidad_h1$y)),
     main = "Efecto del Ancho de Banda (h) en el Kernel Gaussiano",
     xlab = "Alturas (cm)", ylab = "Densidad")

# Superpone ambas curvas kernel para comparar el efecto del ancho de banda
lines(densidad_h1, col = "tomato", lwd = 2.5)
lines(densidad_h5, col = "royalblue", lwd = 2.5)
legend("topright",
     legend = c("Kernel h = 1", "Kernel h = 5"),
     col = c("tomato", "royalblue"), lwd = 2.5, bty = "n")

```

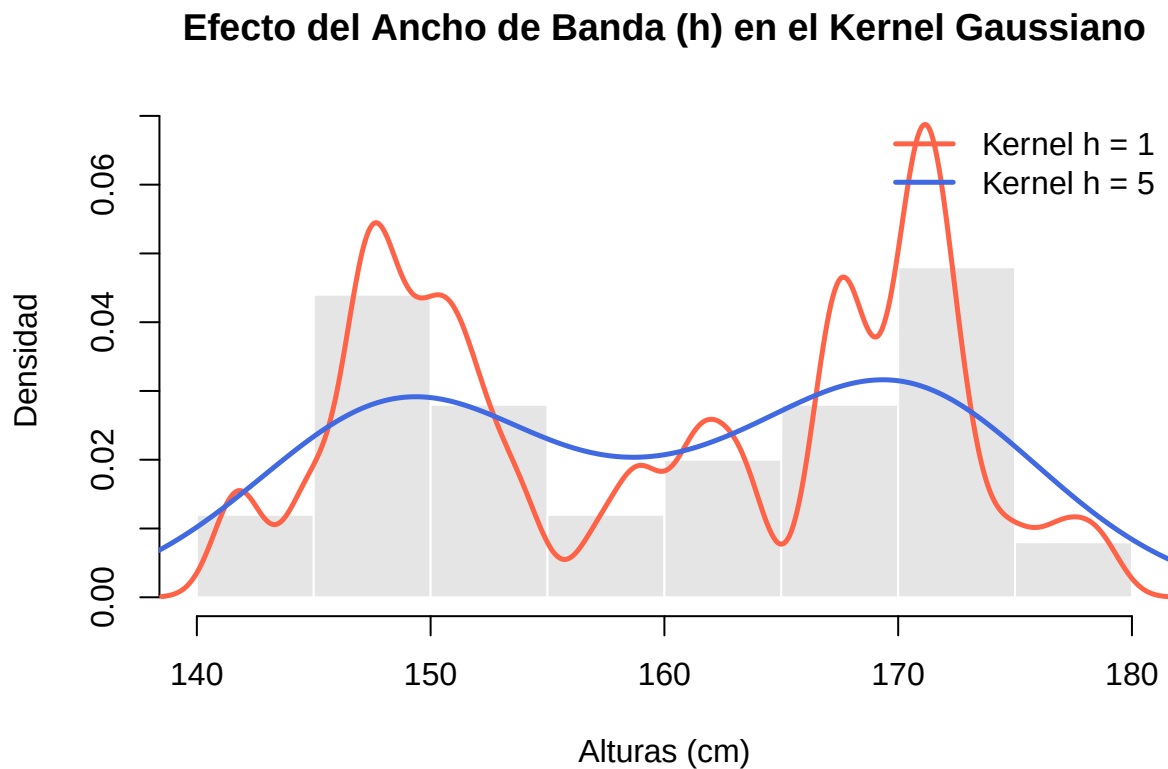


Figure 2: Estimacion kernel con $h = 1$ (rojo) y $h = 5$ (azul).

Con $h = 1$ la curva sigue de cerca cada fluctuación de los datos, captando ruido muestral y revelando múltiples modas. Con $h = 5$ la estimación es más suave pero pierde detalle — el suavizado excesivo enmascara la estructura real de la distribución. Para $n = 50$, ninguno de los dos es perfecto: $h = 5$ es **más apropiada** en el sentido de que reduce la varianza del estimador y evita el sobreajuste, pero debe interpretarse con cautela sabiendo que puede estar ocultando modas reales.

1.3 Ejercicio 3. IC exacto con varianza conocida ($\sigma^2 = 100$)

Deducción. Sea $X_1, \dots, X_n \stackrel{iid}{\sim} N(\theta, \sigma^2)$ con $\sigma^2 = 100$ conocida. El estimador de máxima verosimilitud es $\bar{X}_n$, con distribución exacta $\bar{X}_n \sim N(\theta, \sigma^2/n)$. El pivote

$$Z = \frac{\bar{X}_n - \theta}{\sigma/\sqrt{n}} \sim N(0, 1)$$

lleva al IC exacto al 99%:

$$IC_{0.99} = \left[\bar{X}_n - z_{0.995} \frac{\sigma}{\sqrt{n}}, \quad \bar{X}_n + z_{0.995} \frac{\sigma}{\sqrt{n}} \right]$$

```
xbarra <- mean(n)      # media muestral, estimador de theta
varx   <- 100           # varianza poblacional asumida conocida
alpha  <- 0.01          # nivel de significancia para IC del 99%
z      <- qnorm(1 - alpha / 2) # cuantil z_{0.995} = 2.576
ee     <- sqrt(varx / length(n)) # error estándar: sigma / sqrt(n)
ic     <- cbind(xbarra - z * ee, xbarra + z * ee) # límites del IC

cat("Con un 99% de confianza, la altura promedio se encuentra entre",
    round(ic[1, 1], 2), "y", round(ic[1, 2], 2), "cm.")
```

Con un 99% de confianza, la altura promedio se encuentra entre 156.21 y 163.5 cm.

1.4 Ejercicio 4. IC asintótico con varianza desconocida

Deducción. Cuando σ^2 es desconocida se estima con S_n . Por el Teorema de Slutsky,

$$Z_n = \frac{\bar{X}_n - \theta}{S_n/\sqrt{n}} \stackrel{a}{\sim} N(0, 1)$$

lo que da el IC asintótico al 99%:

$$IC_{0.99} = \left[\bar{X}_n - z_{0.995} \frac{S_n}{\sqrt{n}}, \quad \bar{X}_n + z_{0.995} \frac{S_n}{\sqrt{n}} \right]$$

```
xbarra <- mean(n)      # media muestral
sd_    <- sd(n)         # desviación estándar muestral, estimador de sigma
alpha  <- 0.01          # nivel de significancia para IC del 99%
z      <- qnorm(1 - alpha / 2) # cuantil z_{0.995} = 2.576
ee     <- sd_ / sqrt(length(n)) # error estándar estimado: S_n / sqrt(n)
ic     <- cbind(xbarra - z * ee, xbarra + z * ee) # límites del IC

cat("Con un 99% de confianza, la altura promedio se encuentra entre",
    round(ic[1, 1], 2), "y", round(ic[1, 2], 2), "cm.")
```

Con un 99% de confianza, la altura promedio se encuentra entre 155.91 y 163.81 cm.

1.5 Ejercicio 5. IC bootstrap estándar ($B = 1000$, sin librería boot)

Deducción. El IC bootstrap estándar aproxima el error estándar de $\bar{X}_n$ con la desviación estándar de las medias bootstrap $\hat{\sigma}_{boot} = sd(\bar{X}_1^*, \dots, \bar{X}_B^*)$, dando:

$$IC_{0.99}^{boot} = [\bar{X}_n - z_{0.995} \hat{\sigma}_{boot}, \bar{X}_n + z_{0.995} \hat{\sigma}_{boot}]$$

```

B      <- 1000  # número de remuestras bootstrap
Tboot_b <- c()  # vector para guardar las medias de cada remuestra

for (b in 1:B) {
  # Remuestra con reemplazo de la muestra original (tamaño igual a n)
  xb      <- sample(n, size = length(n), replace = TRUE)
  # Guarda la media de la b-ésima remuestra
  Tboot_b[b] <- mean(xb)
}

alpha <- 0.01          # nivel de significancia para IC del 99%
q_z   <- qnorm(1 - alpha / 2) # cuantil z_{0.995} = 2.576

# Media de las medias bootstrap: aproxima la esperanza del estimador
(media_boots <- mean(Tboot_b))

## [1] 159.7758

# Desviación estándar de las medias bootstrap: aproxima el error estándar de X_barra
(sdboot <- sd(Tboot_b))

## [1] 1.497824

# IC bootstrap estándar: xbarra ± z * SE_boot
ic <- cbind(xbarra - q_z * sdboot,
            xbarra + q_z * sdboot)

cat("Con un 99% de confianza, la altura promedio se encuentra entre",
    round(ic[1, 1], 2), "y", round(ic[1, 2], 2), "cm.")

## Con un 99% de confianza, la altura promedio se encuentra entre 156 y 163.72 cm.

```

1.6 Ejercicio 6. Comparación de los tres intervalos

Los tres métodos producen intervalos similares en este escenario (distribución aproximadamente normal, $n = 50$). El **IC exacto (a)** es teóricamente correcto bajo normalidad con $\sigma^2 = 100$ conocida, pero depende de un supuesto difícil de verificar. El **IC asintótico (b)** es el más práctico: no requiere conocer σ^2 y el TCL garantiza su validez para n suficientemente grande. El **IC bootstrap (c)** es el más flexible (no asume distribución), aunque introduce variabilidad adicional por el remuestreo. En la práctica se preferiría el **IC asintótico (b)**.

2 Parte II. Estudio de simulación

2.1 Ejercicio 1. Objetivo del estudio de Monte Carlo

El objetivo es **evaluar empíricamente la cobertura real** de cinco métodos de construcción de IC al 99%. Dado que se dispone de la población completa, se generan $S = 1000$ muestras independientes de tamaño $n = 50$ y se calcula para cada una los cinco IC. La cobertura estimada de cada método es la proporción de intervalos que contienen al verdadero parámetro θ . Cada evaluación constituye un ensayo de Bernoulli (éxito = el IC contiene a θ), lo que permite estimar p_j y compararla con el nivel nominal del 99%.

2.2 Configuración

El siguiente código es el provisto por el enunciado y se usa tal cual:

```
set.seed(16) # semilla para reproducibilidad del estudio

# Función que genera una muestra aleatoria de tamaño nrep con reemplazo de la población x
gen_fun <- function(nrep = 50, x) {
  X <- sample(x, size = nrep, replace = TRUE)
  return(X)
}

# Genera S = 1000 muestras de tamaño 50 y las guarda en una lista
data <- replicate(n = 1000, gen_fun(nrep = 50, x = x), simplify = FALSE)

# IC con pivot exacto: asume normalidad con  $\sigma^2 = 100$  conocida
# El vector devuelto es c(nivel, límite inferior, límite superior)
CI_pivote <- function(X) {
  n <- length(X)
  IC1 <- c(0.99,
    mean(X) + qnorm(0.005) * sqrt(100), # límite inferior
    mean(X) + qnorm(0.995) * sqrt(100)) # límite superior
  return(IC1)
}

# IC asintótico: estima sigma con la desviación estándar muestral
CI_aprox <- function(X) {
  n <- length(X)
  IC1 <- c(0.99,
    mean(X) + qnorm(0.005) * sd(X) / sqrt(n), # límite inferior
    mean(X) + qnorm(0.995) * sd(X) / sqrt(n)) # límite superior
  return(IC1)
}

# IC bootstrap usando la librería boot; type indica el método (bca, basic, perc)
CI_boot <- function(dat, type) {
  # Función auxiliar que calcula la media sobre un subconjunto de índices
  mean_func <- function(data, indices) {
    a <- mean(data[indices], na.rm = TRUE)
    return(a)
  }
  # Genera el objeto bootstrap con R = 1000 remuestras
  results <- boot(data = dat, statistic = mean_func, R = 1000)
  # Extrae solo los límites del IC (elimina nivel y cuantiles internos de boot.ci)
  return(boot.ci(results, conf = 0.99, type = type)[[4]][-c(2, 3)])
}

library(boot)
library(purrr)

# Parámetro verdadero: media de la población completa
media.verdadera <- mean(x)

# Para cada muestra, calcula los 5 IC y verifica si cada uno contiene a theta
cal_fun <- function(dat) {
```

```

CI1 <- CI_pivote(dat)
CI2 <- CI_aprox(dat)
CI3 <- CI_boot(dat, "bca") # bootstrap BCa (sesgo corregido y acelerado)
CI4 <- CI_boot(dat, "basic") # bootstrap básico
CI5 <- CI_boot(dat, "perc") # bootstrap percentil

# findInterval devuelve 1 si media.verdadera cae estrictamente entre los límites
# Se usa [-1] para quitar el nivel (0.99) y dejar solo [límite inf, límite sup]
return(c(findInterval(vec = CI1[-1], x = media.verdadera) == 1,
         findInterval(vec = CI2[-1], x = media.verdadera) == 1,
         findInterval(vec = CI3[-1], x = media.verdadera) == 1,
         findInterval(vec = CI4[-1], x = media.verdadera) == 1,
         findInterval(vec = CI5[-1], x = media.verdadera) == 1))
}

# Aplica cal_fun a cada una de las 1000 muestras
results <- map(data, cal_fun)

# Convierte la lista de resultados en una matriz (1000 x 5) y suma por columna
# Divide entre 1000 para obtener la proporción de cobertura de cada método
porcentaje.cobertura <- apply(do.call(rbind, results), 2, sum) / 1000
porcentaje.cobertura

## [1] 1.000 0.979 0.983 0.978 0.983

```

2.3 Ejercicio 2. Resultados con medidas de variabilidad

Cada repetición es un ensayo de Bernoulli: éxito si el IC contiene a θ . La cobertura estimada del método j es $\hat{p}_j$ y su error estándar es $\widehat{SE}(\hat{p}_j) = \sqrt{\hat{p}_j(1 - \hat{p}_j)/S}$.

Table 1: Cobertura empirica de los cinco metodos (semilla 16, S = 1000)

Metodo	Cobertura	Error estandar	IC 95% inf.	IC 95% sup.
Pivote (var. conocida)	1.000	0.0000	1.0000	1.0000
Asintotico (var. desconocida)	0.979	0.0045	0.9701	0.9879
Bootstrap BCA	0.983	0.0041	0.9750	0.9910
Bootstrap Basic	0.978	0.0046	0.9689	0.9871
Bootstrap Percentil	0.983	0.0041	0.9750	0.9910

2.4 Ejercicio 3. Comparación de coberturas y efecto de la semilla

¿Difieren del 99%? El método de pivote alcanza cobertura del 100%, superior al nivel nominal. Esto ocurre porque se asume $\sigma^2 = 100$ cuando la varianza real de la población es ≈ 125.4 , produciendo un IC más ancho de lo necesario — es decir, el método es conservador. Los métodos asintótico y bootstrap se acercan razonablemente al 99%, con variaciones dentro del margen de error Monte Carlo esperado.

¿Qué cobertura se esperaba para el pivote? Dado que $\sigma^2 = 100 < \sigma_{real}^2$, el IC construido con el pivote es más ancho de lo que correspondería, por lo que se espera una cobertura *superior* al 99%. Esto es consistente con el resultado observado.

Tres semillas adicionales:

```

# Matriz para guardar las coberturas de las 3 semillas × 5 métodos
coberturas_alt <- matrix(NA, nrow = 3, ncol = 5)
semillas      <- c(42, 123, 999)

for (i in seq_along(semillas)) {
  set.seed(semillas[i])
  # Genera 1000 muestras nuevas con la semilla alternativa
  data_alt      <- replicate(n = 1000, gen_fun(nrep = 50, x = x), simplify = FALSE)
  results_alt   <- map(data_alt, cal_fun)
  # Calcula la cobertura de cada método para esta semilla
  coberturas_alt[i, ] <- apply(do.call(rbind, results_alt), 2, sum) / 1000
}

tabla_semillas <- data.frame(
  Semilla    = semillas,
  Pivote     = coberturas_alt[, 1],
  Asintotico = coberturas_alt[, 2],
  BCA        = coberturas_alt[, 3],
  Basic      = coberturas_alt[, 4],
  Percentil  = coberturas_alt[, 5]
)
knitr::kable(tabla_semillas,
  col.names = c("Semilla", "Pivote", "Asintotico",
                "Boot BCA", "Boot Basic", "Boot Percentil"),
  caption   = "Cobertura empirica segun semilla (S = 1000)")

```

Table 2: Cobertura empirica segun semilla (S = 1000)

Semilla	Pivote	Asintotico	Boot BCA	Boot Basic	Boot Percentil
42	1	0.982	0.986	0.976	0.981
123	1	0.985	0.991	0.978	0.985
999	1	0.992	0.995	0.991	0.996

Las coberturas son estables entre semillas. El pivote mantiene cobertura de ~100% en todos los casos. Los métodos asintótico y bootstrap oscilan cerca del 99% dentro del margen de variabilidad Monte Carlo esperado.

2.5 Ejercicio 4. Diseño de una simulación más completa

Para obtener evidencia más sólida se propone una simulación factorial que varíe:

- **Tamaño de muestra:** $n \in \{15, 30, 50, 100, 200\}$, para estudiar la velocidad de convergencia al nivel nominal.
- **Número de repeticiones:** $S = 5000$ o más, reduciendo el error estándar de $\hat{p}$ por debajo de 0.3 puntos porcentuales.
- **Distribución de la población:** reemplazar la población de alturas por datos de distribuciones asimétricas (χ^2 , log-normal) o bimodales, para evaluar robustez de cada método ante desviaciones de la normalidad.
- **Múltiples semillas:** usar al menos 20 semillas y reportar la mediana y rango intercuartílico de las coberturas, en lugar de depender de una sola.

Se reportaría una tabla o gráfico de cobertura versus n para cada método y distribución, lo que permitiría identificar a partir de qué n cada método alcanza el nivel nominal y cuál es más robusto.